

Institut für Engineering von Produkten und Systeme (IEPS)

Fakultät für Maschinenbau

Computer Vision-Based Pick-and-Place Fischertechnik Crane and Inventory Management

Supervisor:

Jun.-Prof. Dr.-Ing. Sebastian Lang

Msc. Vasu Dev Mukku

Summer Term 2026

Sourya Shome

Course of studies: M.Sc. Digital Engineering

Matriculation number: 252138

E-Mail: sourya.shome@st.ovgu.de

Tanvi Rajesh Tarkunde

Course of studies: M.Sc. Digital Engineering

Matriculation number: 253841

E-Mail: tanvi.tarkunde@st.ovgu.de

Abstract

Modern warehouse logistics demand high levels of accuracy, efficiency, and scalability, but existing systems are still based on a semi-automation approach, with computer vision only being used in detecting objects but not controlling their manipulation. The proposed project is about designing an automated warehouse logistics system that utilizes computer vision combined with control by a microcontroller and simulation validation. The proposed system combines a YOLO-based detection pipeline with a 3-axis gantry mechanism controlled via a Raspberry Pi. Detected objects are mapped from image space to workspace coordinates through calibration, enabling accurate localization and execution of pick-and-place tasks.

A simulation environment is used to validate system behavior prior to deployment, while a lightweight backend provides real-time monitoring and inventory tracking. Experimental results demonstrate reliable detection performance and near real-time operation. Overall, the system highlights how integrating perception with control can improve efficiency, accuracy, and scalability in automated warehouse workflows.

Table of Contents

1	Introduction	4
2	Related Work	5
2.1	Real-Time Object Detection	5
2.2	Robotic Pick-and-Place Automation	6
2.3	Simulation and Digital Twins	6
3	Task Description and Project Goals	6
4	Scientific/Technical Status at Project Start	7
5	Project Plan	8
5.1	Task Breakdown	8
6	Project Execution and Scientific/Technical Results	9
6.1	Hardware Architecture	9
6.2	User Interface and Dashboard Implementaion	11
6.3	Cartesian Pick-and-Place Robot Simulation in CoppeliaSim	12
6.4	Object Detection: Dataset Construction, Training, and Evaluation	16
6.4.1	Dataset Construction and Annotation	16
6.4.2	Data Augmentation	17
6.4.3	Training Protocol	17
6.4.4	Evaluation Metrics	18
6.4.5	Convergence Behaviour and Discussion	19
6.4.6	Deployment via ONNX	20
6.5	Full-Stack System Architecture and Data Flow	20
6.5.1	Architectural Stages	21
6.5.2	End-to-End Control and Data Flow	21
6.5.3	Design Rationale	22
7	Comparison of Project Plan and Execution	23
8	Usability of the Results	24

9	Future Improvements	25
10	Conclusions	25
11	Source Code Repository	26

1 Introduction

The modern logistics process is impossible without efficient warehouse systems, which perform such operations as storage, sorting, and transportation of goods through sophisticated supply chains. Given the increased demands for fast delivery and higher productivity, there is an increased need to perform all these processes accurately, efficiently, and effectively. Automation plays a vital role in modern logistics by reducing repetitive manual tasks and improving operational efficiency.

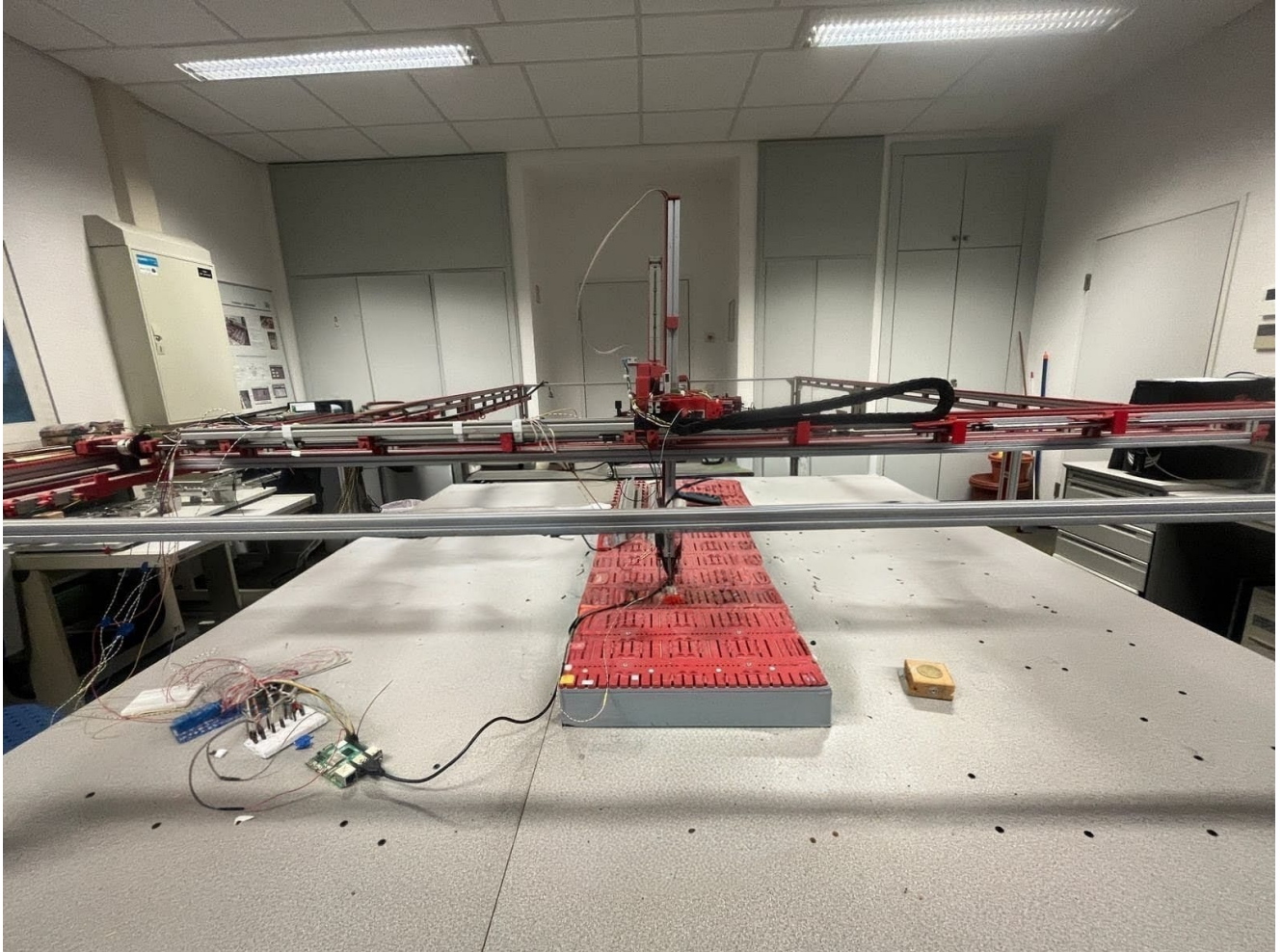


Figure 1: Pick and place Fischertechnik model crane

Figure [9] shows the experimental configuration that is employed in this study. This robotic system is implemented using a Fischertechnik model crane, and is essentially a Cartesian robot that can move about three orthogonal planes (X, Y, and Z). These types of robots work within a confined work environment and allow precision movement and positioning; consequently, they find application in pick-and-place operations in warehouses. The experimental configuration consists of a vision system for perception, the work area, and actuators to perform actions.

Even in automated warehouses, the processes of item selection, sorting, and moving can be either manually done or semiautomated. Even with the introduction of robotic systems to help perform these activities, the efficiency of these systems is hindered by the fact that they lack the coordination between perception and action. Computer vision systems can effectively detect and track objects; however, the

detected information is often not directly integrated with robotic actuation.

Conventional methods for automating warehouses have several limitations. There may be a need for manual inputs in interpreting the findings of the detection process, entering data in the control system, or checking the process itself. These flaws translate into higher latencies, less efficiency, and more possibilities for errors, especially under high-volume situations. In addition, there are no real-time checks and monitoring systems that would make sure of the efficiency and accuracy of the process.

There is an increasing demand for systems that would be able to recognize, locate, and manipulate objects with constant situational awareness of their surroundings (1). These kinds of systems need to be capable of translating visual information into accurate spatial information and performing physical actions within a work space.

The recent developments in robotics and computer vision technologies have facilitated the use of automated pick-and-place mechanisms, which are intended to move items from one point to another with utmost accuracy and consistency (2). In particular, gantry type robots, which work on three mutually perpendicular axes (X-Y-Z), have been widely adopted in the industrial sector owing to their accuracy and scalability. This makes the use of such systems ideal for automated warehousing implementations.

The current project is intended to bridge the gap between visual recognition and execution by creating an autonomous pick-and-place system based on the application of computer vision techniques to the Fischertechnik crane system. Object recognition, spatial positioning, and manipulation are achieved through a single integrated platform allowing for automated picking up, stowing away, and dispatching objects. Additionally, the use of CoppeliaSim software is applied to simulate system operations.

This is achieved by automating processes through the application of an intelligent system that is integrated into one seamless loop. Besides increased efficiency, error reduction, and improved system reliability, the solution facilitates the provision of more informed decisions due to continuous monitoring and analysis of data.

2 Related Work

The system developed in this project draws on three established research areas: real-time object detection, robotic pick-and-place automation, and simulation-based validation through digital twins. This section briefly reviews the most relevant contributions in each.

2.1 Real-Time Object Detection

Object detection has been dominated by two competing paradigms. Two-stage detectors such as Faster R-CNN (3) first generate region proposals and then classify them, achieving high accuracy at the cost of inference speed. Single-stage detectors instead cast detection as a unified regression problem over a dense grid, trading a small accuracy margin for substantially lower, more deterministic latency. The You Only Look Once (YOLO) family pioneered this approach (4) and has since undergone continual refinement

through successive generations (4, 5, 6, 7), with the Single Shot MultiBox Detector (8) representing a parallel line of single-shot work. Modern variants such as Ultralytics YOLO11 (9) incorporate anchor-free heads, distribution focal loss (10), and CIoU-based bounding-box regression (11) atop deep residual backbones (12). The deterministic, low-latency inference offered by single-stage detectors makes them especially suited to closed-loop robotic control, motivating their selection in this work.

2.2 Robotic Pick-and-Place Automation

Pick-and-place is among the most widely studied tasks in industrial robotics, with Cartesian gantry robots favoured in warehousing for their accuracy, repeatability, and scalability (2). The rapid growth of e-commerce has further accelerated innovation in warehouse robotics, driving demand for low-cost, perception-guided material-handling systems (13). A recurring limitation in prior prototype systems is the weak coupling between perception and actuation: vision modules detect and localize objects, but the detected information is not always propagated directly into motion commands. The present work specifically targets this gap by mapping detector outputs into workspace coordinates that drive the gantry actuators.

2.3 Simulation and Digital Twins

Simulation environments allow control logic, kinematics, and grasping behaviour to be validated before deployment on physical hardware, reducing development risk. CoppeliaSim, originally introduced as V-REP (14), provides a versatile and scalable framework for modelling articulated and Cartesian robots. More broadly, the digital-twin paradigm—maintaining a synchronized virtual replica of a physical system—has gained significant traction in Industry 4.0 manufacturing and warehousing (15, 1). This project employs CoppeliaSim as an offline validation environment and identifies real-time hardware–simulation synchronization as a direction for future work.

3 Task Description and Project Goals

The project aims to develop a prototype automated warehouse system using a Fischertechnik crane by addressing several critical objectives:

- Design and implement a small-scale system capable of object detection, electromagnetic pickup, and autonomous object placement at desired points to demonstrate realistic warehouse operation.
- Use a microcontroller along with relay modules for controlling DC motors for 3-axis motion ensuring basic positional control and repeatable movement within the prototype setup.
- Establish a pipeline that links the detection of objects to relevant motor actions, highlighting the relationship between perception and actuation.

- Develop a simplified simulation of the prototype system to test motion control and actuation capabilities prior to implementation in real-life conditions.
- Implement an intuitive interface for visualizing the state of the prototype system and the management of objects via said system.

4 Scientific/Technical Status at Project Start

At the beginning of the project, the scientific and technical foundation was characterized by the growing need to bridge the gap between computer vision-based perception and physical actuation in warehouse automation systems. While object detection techniques using deep learning models such as YOLO were already well-established (3), their integration with real-time control systems for autonomous pick-and-place operations remained limited, particularly in low-cost and educational prototype environments (4). Resources accessible during project start include:

- **Data Acquisition and Processing :** Visual data was acquired using a camera module connected to a Raspberry Pi, thus enabling live visual feed from the working space. For the initial datasets needed for object detection, manual collection and labeling were employed to train the YOLO model.
- **Control System :** The Raspberry Pi-based embedded system was used to interact with the relay module for physical controls. The control system facilitated:
 - Movement of DC motors in X, Y, and Z directions.
 - Directional change using relay-based logic.
 - Electromagnet activation for picking and placing objects.

This served as an inexpensive and versatile control mechanism for robotic movements without precision feedback devices like encoders.

- **Integration with Computer Vision :** YOLO-based object detection was used to detect objects in the environment. The system was capable of:
 - Detecting objects and generating bounding boxes.
 - Providing confidence scores for predictions.
 - Conduct inference on the edge with acceptable delay.

But still at this point, the system did not have precise calibration and transformation from pixels coordinates to real world coordinates.

- **Simulation Environment :** The gantry robot's 3-axis system was modeled using CoppeliaSim. The environment had:

- Prismatic joint modeling the X, Y, and Z axes.
- A basic electromagnet end effector.
- Primitive movement control for pick and place operations testing.

This simulation environment facilitated the early validation of movement controls, although synchronization is yet to be achieved.

- **Backend and Monitoring System Architecture:** A lightweight backend system utilizing FastAPI was employed for system communication and data flow. The Streamlit frontend provided:
 - Real-time visualization of system status.
 - Monitoring of object detection and actions.
 - Inventory tracking capability.

Although suitable for demonstrations, the system was lacking certain functionalities such as OCR implementation, anomaly detection, and enhanced real-time performance.

5 Project Plan

This project plan was created to help in developing a system that involves a structured way of designing a pick-and-place computer vision prototype system. It helps to lay down the important steps and tasks that must be considered in the design process. This process took a gradual step-by-step approach due to the feedback obtained during experimentation and other constraints.

5.1 Task Breakdown

Phase 1: Initialization and System Setup

- Planning and Setting Up Resources: Development process workflow, tools, and hardware set up.
- Hardware Set Up: Setting up Raspberry Pi, relay components, motors, and cameras.
- Learning About Tools and Frameworks: Learning about YOLO framework, GPIO in Raspberry Pi, and simulation software like CoppeliaSim.
- Result: Baseline system set up with hardware and development resources.

Phase 2: Image Capture and Object Detection

- Image Data Capture: Take images of the objects in the workspace under various circumstances.
- Data Annotation and Labeling: Annotate images by drawing bounding boxes on the images.
- Object Detection Model Training: Train an object detection model using Ultralytics YOLO11s.

- Inference Pipeline Design: Develop an object detection pipeline that detects objects and provides bounding boxes with probability values.
- Result: Object detection model to detect objects in the workspace.

Phase 3: System Integration

- Coordination of Mapping and Calibration: Translate the coordinates from the images into actual physical positions.
- Motor Control Design: Design the control logic for X, Y, Z movements via Raspberry Pi and relays.
- Object Detection and Motion Integration: Integrate the results of the detection process into controlling the motors for autonomous pick-and-place operations.
- Electromagnet Control: Control electromagnets to perform pick-and-place operations autonomously.

Phase 4: Simulation, Monitoring, and Optimization

- Simulation: Model the gantry system using CoppeliaSim for testing purposes.
- Monitoring System: Design interface with FastAPI and Streamlit to monitor and visualize the data.
- Optimization of Model: Enhance model efficiency through ONNX implementation and dataset augmentation

6 Project Execution and Scientific/Technical Results

6.1 Hardware Architecture

The circuit diagram shown in Figure [2] illustrates the full hardware setup for the project, with the use of a Raspberry Pi as the primary controller that controls a three-axis gantry system using relay-controlled motors. This configuration provides a well-defined distinction between the low-power control signal generation and the high-power motor operation.

The first component at the higher end of this process is the Raspberry Pi board itself, which produces a signal through its General Purpose Input Output (GPIO) interface. The GPIO interface runs on a low voltage (3.3V) and provides a low current output and is thus not powerful enough to drive any high-power elements like DC motors. There is, therefore, a need for a medium between these two ends.

These signals from the GPIO pins connect to two 4-channel relay boards. The relays can be considered electrical switches that allow the current to pass when triggered by the signal coming from the Raspberry Pi. A relay functions as an electrically controlled switch that makes it possible for a low-voltage signal to

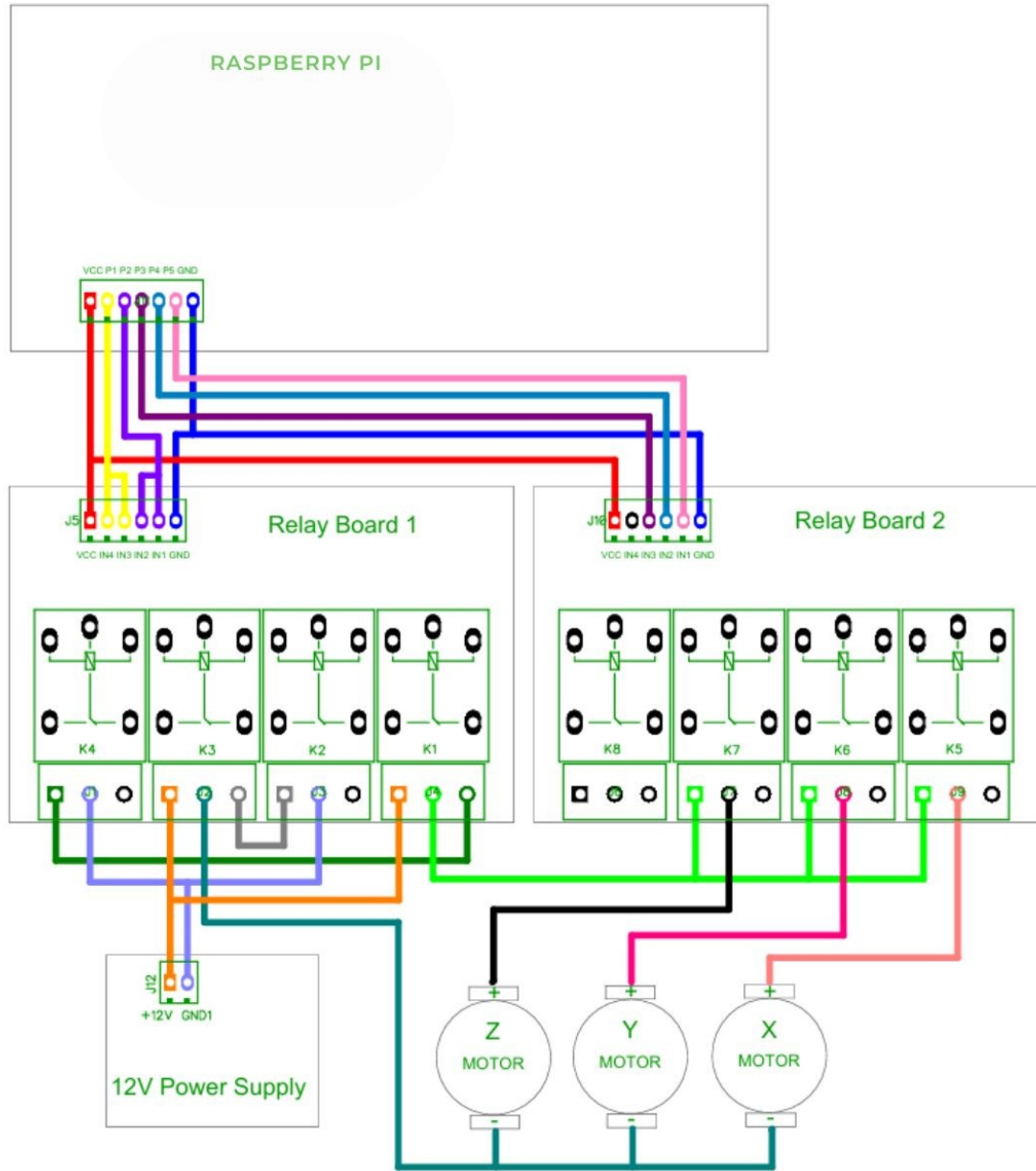


Figure 2: Circuit diagram of Raspberry Pi, relay modules, and motor connections for the pick-and-place system.

turn on a high-voltage circuit without electrical contact between them. This electrical isolation protects the Raspberry Pi from voltage spikes and high-current transients generated by the motors.

The relay modules are powered using an external 12V power source, providing the needed power to drive the DC motors. The relay module is used as a gate for switching on and off the current flowing to the motor from the external power source. Whenever there is a signal from the relay to operate, it closes the circuit between the power source and the motor.

In addition, each motor used in the system (corresponding to the X, Y, and Z axes) is powered by a pair of relays. As a result, polarity control of the voltage fed to the motor becomes possible. Changing polarity leads to changing current flow, thus making it possible to change motor rotation direction. In this way, bidirectional movement is achieved for each axis, enabling full spatial control of the gantry system.

Motors are designed to use relay boards for powering in order to control the direction and operation of

the motion of the motors via relay outputs. The first relay makes the motor rotate in one direction while the second relay makes the motor rotate in the opposite direction. However, both relays being inactive stops the motor. Therefore, such a design of the relay system duplicates the function of the H-bridge circuit. Finally, to guarantee a correct signal transmission from the Raspberry Pi controller to relays, a common ground is required to be maintained in all of them and in the power supply.

The system operates in a manner that ensures that a continuous control is achieved. In particular, the Raspberry Pi obtains the processed data from the computer vision system and analyzes it in order to determine the direction that it has to move. Depending on the analysis, either a HIGH signal or a LOW signal is sent to the GPIO pin that controls the relay. This relay turns ON depending on whether the signal was HIGH or LOW, thereby sending power to the correct motor.

In summary, this circuit works well because it combines digital control with mechanical actuation. Using relay modules ensures that the operation is safe and simple and also gives enough functionality for a prototype but at the same time limits the system by not offering speed control among others.

6.2 User Interface and Dashboard Implementaion

The user interface of the system was developed using the Streamlit framework, which enables the creation of interactive web-based dashboards directly using Python. This approach allows seamless integration between the frontend and backend components without requiring complex web development frameworks.

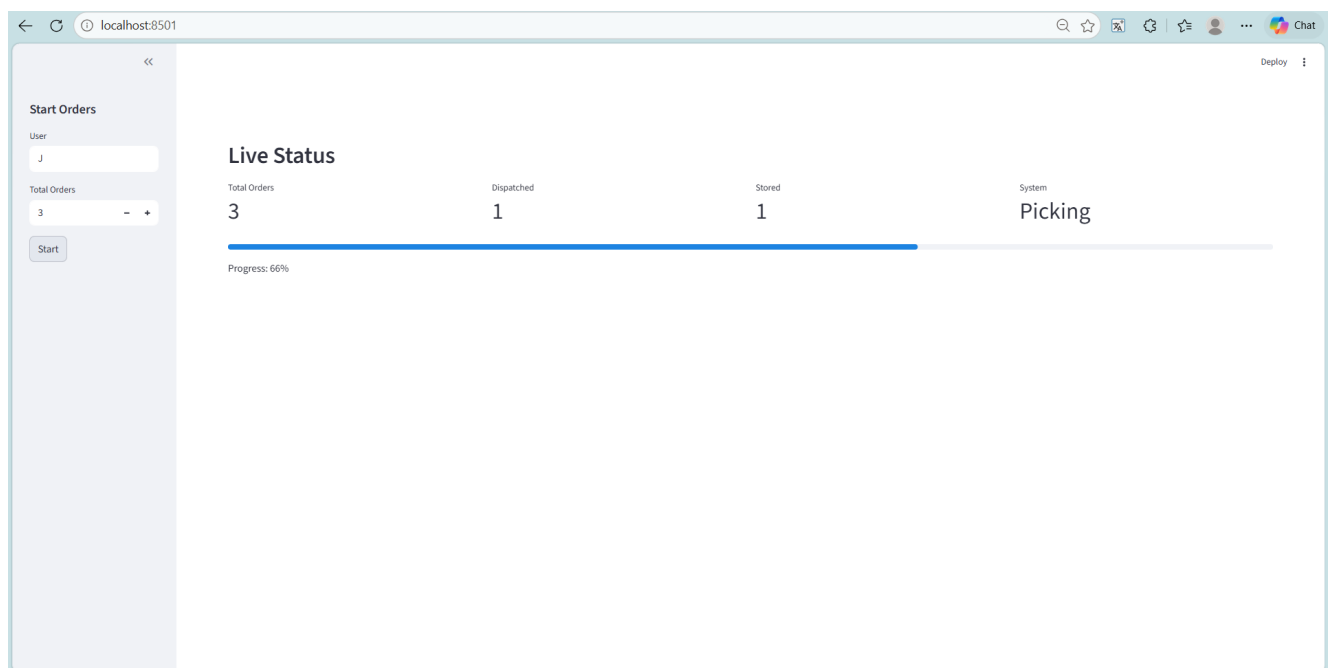


Figure 3: Streamlit-based dashboard for monitoring and controlling the pick-and-place system in real time

The dashboard serves as the primary interface between the user and the automated pick-and-place system. It allows the user to start the ordering process, observe the activities in the system, and view the performance measures. The following Figure [3] represents a summary of the dashboard that was implemented.

The system is divided in two basic elements – the sidebar that is used to interact with the software, and the display area. The sidebar allows users to start the pick-and-place process by entering their name and the number of orders. When the “Start Processing” button is clicked, a request is sent to the backend, and a notification confirms that the process has started.

The main dashboard is organized into tabs such as Overview, History, and Charts, making the interface structured and easy to extend. The Overview tab displays real-time system information, including total orders, dispatched orders, remaining orders, stored objects, and machine status. These values are continuously updated by fetching data from the backend.

A progress bar is used to visually indicate the completion of orders, helping users quickly understand system progress. In addition, notifications are shown when orders are completed and when all tasks are finished, providing clear feedback during operation.

Communication between the frontend and backend is realized by means of HTTP requests, wherein the dashboard gets the system state information and displays it on the screen. The backend stores the shared state and keeps modifying it using a background worker process to ensure consistency between system execution and display of system data. The developed interface demonstrates that lightweight web technologies can effectively support monitoring and control functions.

6.3 Cartesian Pick-and-Place Robot Simulation in CoppeliaSim

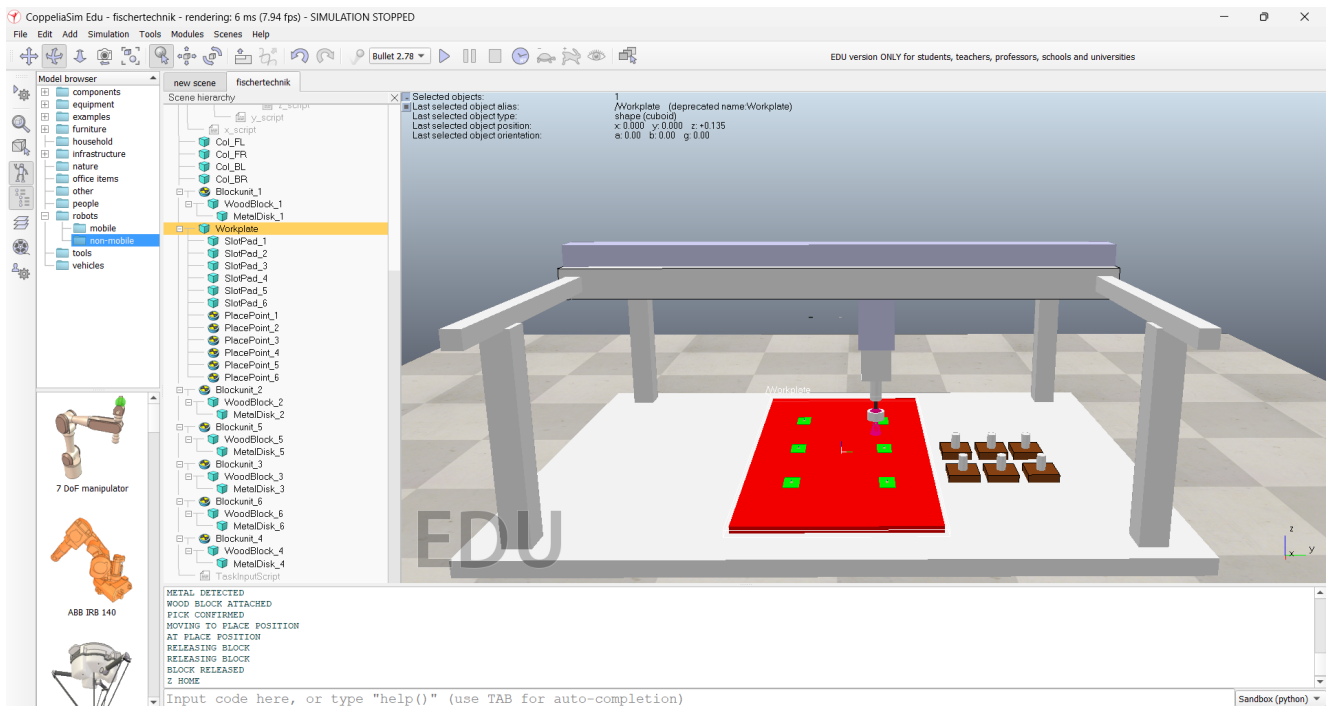


Figure 4: Simulation environment of a three-axis Cartesian (gantry) robot in CoppeliaSim, equipped with an electromagnetic end-effector and proximity sensor, demonstrating pick-and-place operations within a structured workspace.

The developed simulation represents a three-axis Cartesian (gantry) robot designed to perform automated pick-and-place operations within a structured workspace in CoppeliaSim, whose underlying simulation

framework was introduced as V-REP (14). The robot consists of the three perpendicular axes X, Y, and Z, which enables the positioning of the end effector in order to interact with objects that are mounted on the worktable.

The mechanical configuration has a platform base with guiding rods for linear motion in the X-axis direction. This has a moving carriage that allows movement in the Y-axis direction, while a Z-axis slide facilitates vertical motion to interact with the object. Finally, the electromagnet acts as the end effector at the tip of the kinematic chain to facilitate non-contact picking up of the object. A proximity sensor (Magsensor) is attached just below the electromagnet to sense the presence of any object.

Robot motion is controlled through independent joint scripts for the X, Y, and Z axes. As compared to continuous scanning techniques, the system uses state machine-based control methodology. The robot moves towards specific target coordinates that correspond to picking up and placing points chosen by users.

The X-axis and Y-axis are responsible for horizontal positioning, while the Z-axis performs lowering and lifting operations during pickup and placement. The robot follows a coordinated sequence consisting of loading the selected targets, moving toward the object, picking the object using the electromagnet, transporting it to the desired SlotPad, releasing the object, and finally returning to its home position before accepting the next task.

An example of the X-axis positioning logic is shown below:

```
if state=='PICK' then
    target=-pickPos[1]
    sim.setJointTargetPosition(joint,target )

elseif state=='PLACE' then

    target=-placePos[1]
    sim.setJointTargetPosition(joint,target)

end
```

For the gripping mechanism, proximity sensor and dynamic object parenting have been done. The proximity sensor will continuously detect the selected metallic object, and on detection, the respective block gets attached to the electromagnet through the concept of object parenting in CoppeliaSim. During placement, the object is detached from the electromagnet and positioned precisely at the selected SlotPad location.

The following code snippet demonstrates the magnetic attachment logic:

```
local detected=sim.checkProximitySensor(sensor,metalDisk)

if detected > 0 then

    attachedBlock=sim.getObjectParent(metalDisk)
```

```
sim.setObjectParent(attachedBlock, magnet, true)
```

```
attached=true
```

```
sim.setInt32Signal('picked',1)
```

```
end
```

The graphical user interface was created with the simUI module in CoppeliaSim. The GUI allows users to specify the block number to be picked and the target SlotPad for placement. The interface disappears once the user presses the “START TASK” button until the robot completes its task.

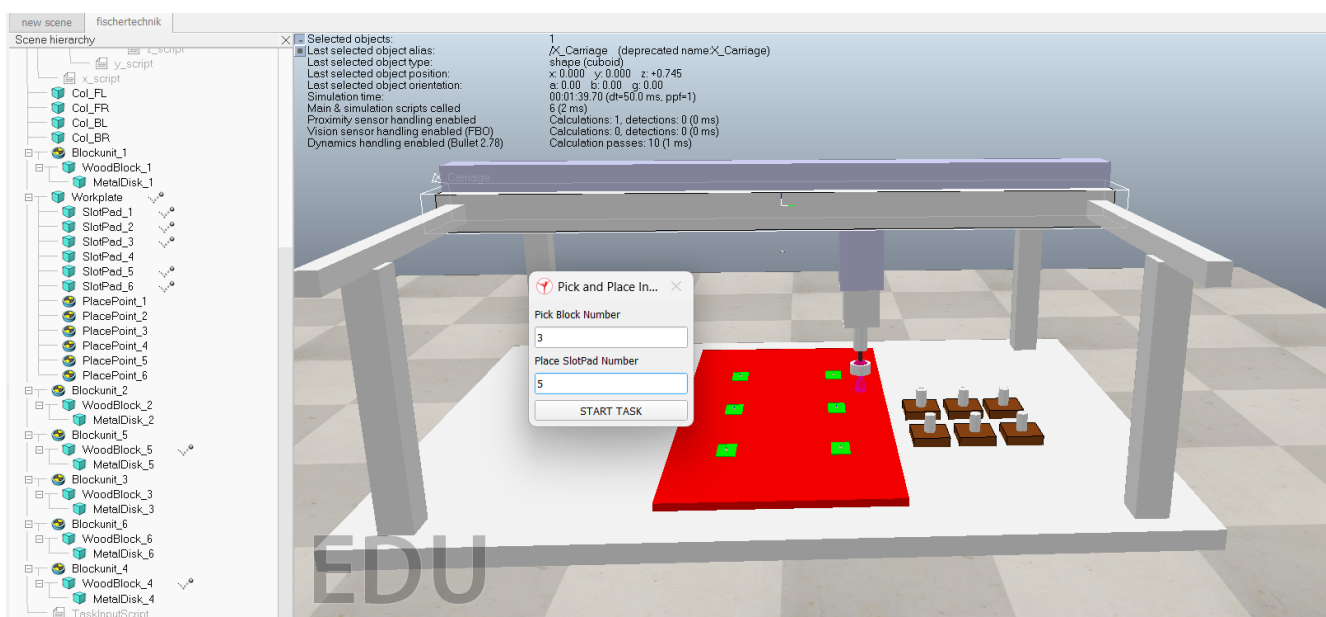


Figure 5: User interface for selecting the block number and target SlotPad.

Additional validation logic was implemented to improve the reliability of the simulation. Two tables named `occupiedSlots` and `placedBlocks` are maintained to track already occupied SlotPads and previously placed blocks.

Before starting a task, the system checks whether:

- the selected SlotPad already contains a block
- the selected block has already been placed

If either condition is violated, a warning popup is displayed and the operation is rejected.

The following logic is used for SlotPad occupancy validation:

```
if occupiedSlots[placeIndex]==true then
```

```
sim.msgBox(
```

```

sim.msgbox_type_warning,
sim.msgbox_buttons_ok,
'SLOT OCCUPIED',
'This SlotPad is already occupied.Please choose another slot.')
```

return

end

Similarly, block reuse prevention is implemented as:

```

if placedBlocks[pickIndex]==true then

sim.msgBox(
sim.msgbox_type_warning,
sim.msgbox_buttons_ok,
'BLOCK ALREADY PLACED',
'This Block has already been placed.
Please choose another block.')
```

return

end

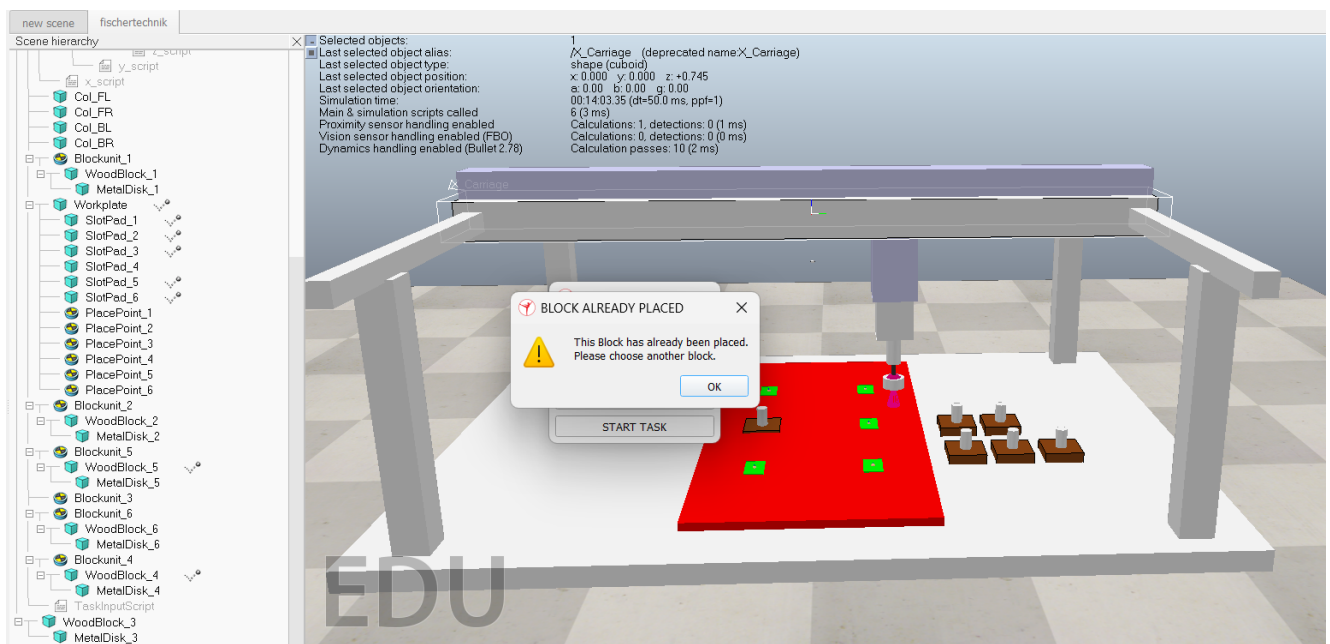


Figure 6: Warning popup displayed when an already placed block is selected again.

In conclusion, the simulation demonstrates important concepts in Cartesian robotic manipulation including coordinated axis control, sensor-based object detection, electromagnetic gripping, state-machine-based task execution, and user-driven pick-and-place automation. The addition of SlotPad occupancy validation and block reuse prevention further improves the realism and reliability of the system for industrial automation applications.



Figure 7: Warning popup displayed when the selected SlotPad is already occupied

6.4 Object Detection: Dataset Construction, Training, and Evaluation

The perception subsystem is responsible for localizing target objects in the workspace and supplying their image-plane coordinates to the coordinate-mapping stage. We adopt the single-stage detector paradigm, which formulates detection as a unified regression problem over a dense grid of anchor-free predictions, thereby achieving the low, deterministic latency required for closed-loop robotic actuation (3). Concretely, we fine-tune the Ultralytics YOLO11s model (9), a compact convolutional architecture comprising 9.41 M parameters and a computational cost of 21.3 GFLOPs at an input resolution of 640×640 . The YOLO11 family continues the line of single-stage detectors that progressively improved the speed–accuracy trade-off (5, 6, 7), in contrast to slower two-stage region-proposal pipelines (?) and earlier single-shot detectors (8). The small variant was selected deliberately: its memory footprint and arithmetic intensity are compatible with edge deployment on the Raspberry Pi 5, while its accuracy remains sufficient for the single-class detection task addressed here.

6.4.1 Dataset Construction and Annotation

A bespoke dataset was assembled in situ to capture the precise illumination, background texture, and camera geometry of the deployment environment, mitigating the domain shift that typically degrades models trained on generic public corpora such as PASCAL VOC (16) and COCO (17). Raw images of the target objects were acquired with the workspace-mounted camera module under varying lighting and pose conditions. Ground-truth bounding boxes were annotated manually in Label Studio and exported in the normalized YOLO format (c, x_c, y_c, w, h) , where c denotes the class index and $(x_c, y_c, w, h) \in [0, 1]^4$ are the box centre and dimensions expressed as fractions of the image width and height. The annotated corpus comprises 100 images and 189 object instances across a single class.

6.4.2 Data Augmentation

Because the labelled corpus is small relative to the parameter count of the network, an offline augmentation pipeline was implemented in Python to enlarge the effective training distribution and regularize the model against overfitting. The augmentation operators comprise:

- **Geometric transformations** — horizontal and vertical flips and in-plane rotations, with bounding-box coordinates transformed consistently to preserve label validity. These encourage invariance to object orientation on the worktable.
- **Photometric transformations** — adjustments to brightness and contrast that emulate the diurnal and shadowing variations observed in the laboratory, improving robustness to illumination change.

By synthesizing multiple variants per source image, the pipeline expands intra-class appearance diversity without additional manual labelling, a standard and effective strategy in the small-data regime (18).

6.4.3 Training Protocol

The network was initialized from COCO-pretrained weights and fine-tuned for 60 epochs on a single NVIDIA Tesla T4 GPU (~14.9 GB) with a mini-batch size of 16 and an input resolution of 640×640 . Transfer learning from large-scale pretraining, enabled by deep residual backbones (12), is essential for convergence on the limited in-domain corpus. Optimization used the AdamW algorithm (19), a decoupled-weight-decay variant of Adam (20), with an initial learning rate of 2×10^{-3} . The composite training objective is the standard YOLO multi-task loss,

$$\mathcal{L} = \lambda_{\text{box}} \mathcal{L}_{\text{box}} + \lambda_{\text{cls}} \mathcal{L}_{\text{cls}} + \lambda_{\text{dfl}} \mathcal{L}_{\text{dfl}}, \quad (1)$$

where $\mathcal{L}_{\text{box}}$ is a CIoU-based localization term (11), $\mathcal{L}_{\text{cls}}$ is a binary cross-entropy classification term, $\mathcal{L}_{\text{dfl}}$ is the distribution focal loss (10) that refines bounding-box regression by modelling box edges as discretized probability distributions, and $\lambda_{(\cdot)}$ are the corresponding weighting coefficients. Total wall-clock training time was 0.186 h (~11 min). The hyperparameter configuration and runtime characteristics are consolidated in Table 1.

Parameter	Value	Parameter	Value
Backbone	YOLO11s (COCO-pretrained)	Epochs	60
Parameters	9.41 M (21.3 GFLOPs)	Optimizer	AdamW ($\eta_0 = 2 \times 10^{-3}$)
Dataset	100 images / 189 instances	Hardware	NVIDIA Tesla T4 (~14.9 GB)
Classes	1	Train time	0.186 h (~11 min)
Input size	640×640	Inference	5.3 ms / image (188 FPS)
Batch size	16	Export	ONNX, best.onnx (36.6 MB)

Table 1: Training configuration and runtime characteristics of the fine-tuned YOLO11s detector.

6.4.4 Evaluation Metrics

Detector quality was assessed using the standard object-detection metrics. Let TP, FP, and FN denote true positives, false positives, and false negatives at a given intersection-over-union (IoU) matching threshold. Precision and recall are defined as

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad (2)$$

where precision quantifies the fraction of detections that are correct and recall the fraction of ground-truth objects recovered. The mean Average Precision (mAP) integrates the precision–recall curve over the recall domain,

$$AP = \int_0^1 P(R) dR, \quad mAP = \frac{1}{N_c} \sum_{i=1}^{N_c} AP_i, \quad (3)$$

with N_c the number of classes. We report mAP@50 (IoU threshold 0.50) and the stricter mAP@50–95, averaged over IoU thresholds from 0.50 to 0.95 in steps of 0.05 following the COCO evaluation protocol (17), which more severely penalizes localization error. The final validation results are reported in Table 2.

Metric	Value	Description
Precision	0.885	Fraction of detections that are correct
Recall	0.984	Fraction of ground-truth objects detected
mAP@50	0.977	Mean AP at IoU = 0.50
mAP@50–95	0.974	Mean AP averaged over IoU $\in [0.50, 0.95]$

Table 2: Validation performance of the fine-tuned YOLO11s detector. The peak mAP@50–95 of 0.974 was obtained at epoch 57.

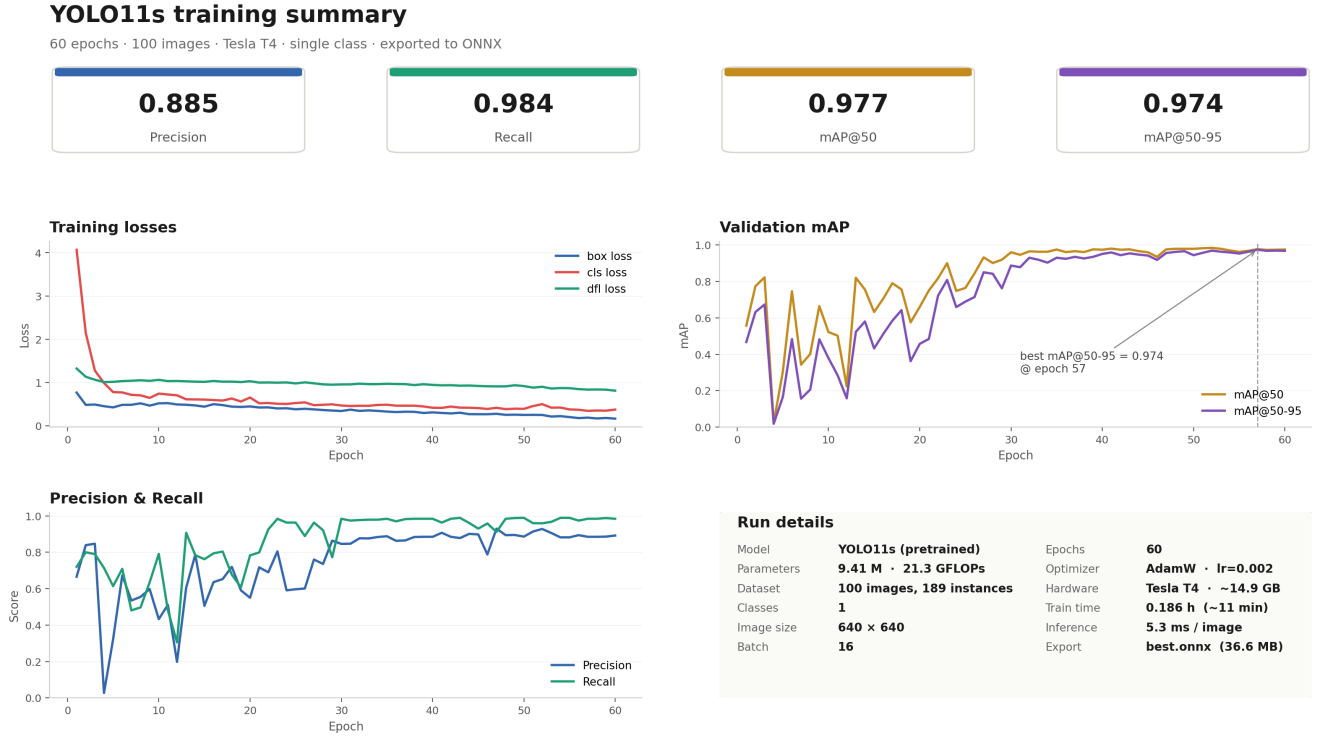


Figure 8: YOLO11s training summary: convergence of the training losses (box, classification, and distribution focal loss) and the validation precision, recall, mAP@50, and mAP@50–95 over 60 epochs.

6.4.5 Convergence Behaviour and Discussion

All three loss components in Equation (1) decreased monotonically and converged within the first ~30 epochs (Figure 8), after which validation mAP plateaued near its asymptote; the best checkpoint (mAP@50–95 = 0.974) was recorded at epoch 57, indicating stable optimization without late-stage divergence. The near-saturated recall of 0.984 confirms that the detector misses very few ground-truth instances, a property of primary importance for the downstream pick-and-place pipeline, in which an undetected object cannot be retrieved. The comparatively lower precision of 0.885 reflects a modest false-positive rate; in the present single-class, structured-workspace setting these spurious detections are readily suppressed by a confidence threshold and geometric gating against the known worktable extent. The marginal gap between mAP@50 (0.977) and mAP@50–95 (0.974) further demonstrates that localization remains accurate under tightened IoU criteria, which is essential because bounding-box

centroids are propagated directly into the image-to-workspace coordinate transformation. We note that the dataset is small and single-class, so the reported metrics characterize the controlled deployment environment rather than open-world generalization; broadening the class taxonomy and corpus size is identified as future work in Section 9 (cf. 4).

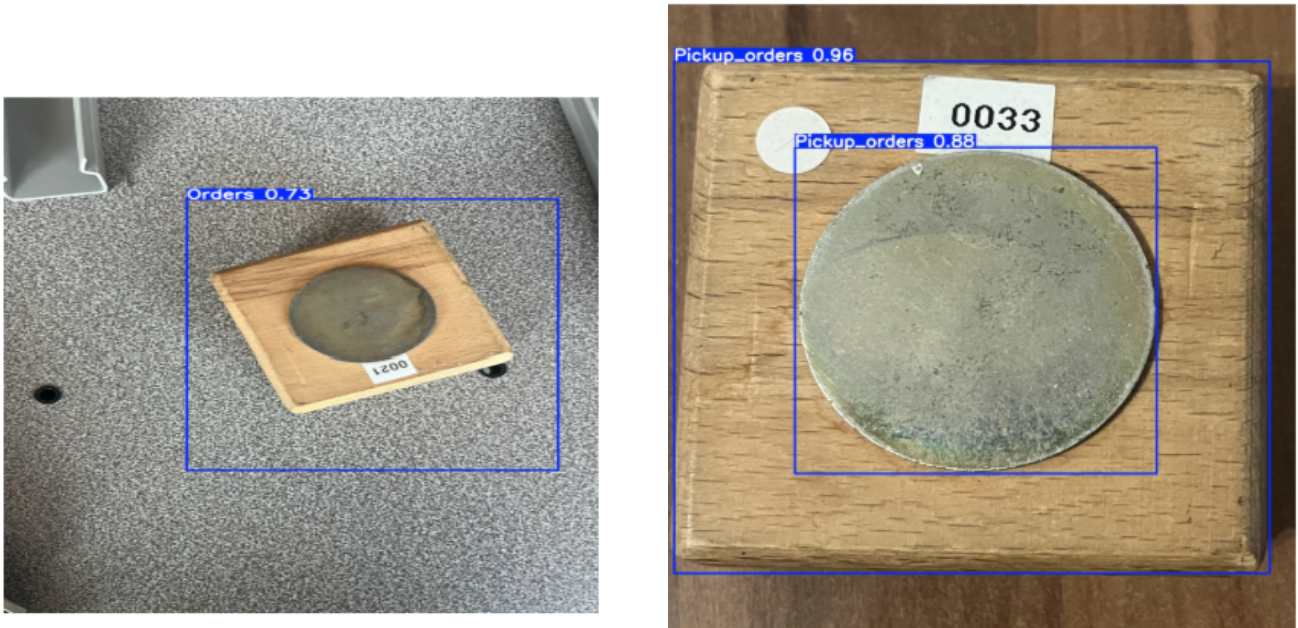


Figure 9: Object detection with confidence scores

6.4.6 Deployment via ONNX

For inference, the best-performing checkpoint was exported to the Open Neural Network Exchange (ONNX) format (21), yielding a self-contained 36.6 MB graph that decouples the model from the training framework and enables execution through optimized runtimes. On the target hardware this configuration achieves a per-image inference latency of approximately 5.3 ms, comfortably within the timing budget of the actuation loop and leaving ample headroom for the concurrent OCR and control workloads described below.

6.5 Full-Stack System Architecture and Data Flow

The complete system is organized as a layered, loosely coupled service architecture that separates the human-facing presentation layer, an orchestration and processing layer, the perception layer, and the physical actuation layer. This separation of concerns follows established software-engineering practice: each stage exposes a well-defined interface, communicates through asynchronous message passing, and can be developed, tested, and substituted independently. Figure ?? depicts the end-to-end pipeline and the direction of data flow between stages.

6.5.1 Architectural Stages

- **Presentation layer — Dashboard (Streamlit).** A reactive, browser-based interface (22) through which the operator submits orders and observes live system state. Rendering is fully decoupled from business logic: the dashboard holds no authoritative state of its own and instead polls the backend, ensuring that the displayed view is always a faithful projection of the canonical server state.
- **Service layer — REST API (FastAPI).** An asynchronous ASGI service (23) that exposes the system’s operations as HTTP endpoints. It validates and accepts incoming order requests, returns immediately to keep the interface responsive, and delegates long-running execution to the worker, thereby implementing a non-blocking request/response contract.
- **Orchestration layer — Asynchronous worker (background thread).** A decoupled execution context that dequeues accepted orders and drives the perception–actuation sequence to completion. The worker owns and mutates the shared system state under controlled access, decoupling task duration from the request lifecycle and preventing the user interface from stalling during physical operations.
- **Perception layer — Detection (YOLO11/ONNX).** The exported detector (Section 6.4) localizes candidate objects and emits bounding boxes that are subsequently transformed from image coordinates to workspace coordinates for grasp planning.
- **Perception layer — Optical character recognition (EasyOCR).** Serial identifiers printed on each object are read with EasyOCR (24), whose recognition stage builds on convolutional-recurrent sequence models for scene text (?), associating every physical item with a unique logical key. This binding is what elevates the system from blind pick-and-place to genuine inventory management, enabling traceable storage and retrieval.
- **Actuation layer — Robot (Fischertechnik gantry).** The three-axis Cartesian manipulator executes the commanded motion and electromagnetic grasp, closing the loop between perceived intent and physical effect.

6.5.2 End-to-End Control and Data Flow

A complete transaction proceeds as a directed pipeline. The operator submits an order through the dashboard; the FastAPI service validates the request, persists it to the shared state, and acknowledges immediately; the asynchronous worker then dequeues the task and invokes the perception layer, where the YOLO11 detector localizes the target object and EasyOCR resolves its serial identifier; the resulting target pose is mapped to workspace coordinates and dispatched as motion commands to the gantry, which performs the pick-and-place operation; finally, the worker updates the shared state with the new inventory and order status, which the dashboard reflects on its next poll. Formally, the flow can be summarized as

$$\text{Dashboard} \rightarrow \text{API} \rightarrow \text{Worker} \rightarrow \text{Detection} \rightarrow \text{OCR} \rightarrow \text{Robot} \rightarrow \text{State update}, \quad (4)$$

with state propagating back to the dashboard to close the supervisory loop.

6.5.3 Design Rationale

This modular decomposition confers several engineering advantages relevant to both research and industrial practice. The asynchronous boundary between the API and the worker guarantees user-interface responsiveness irrespective of mechanical actuation latency. The framework-agnostic ONNX detector can be retrained or upgraded without modifying any other stage. The single shared-state authority eliminates view inconsistencies between the operator display and the physical system. Finally, the clean interfaces between layers render the architecture horizontally extensible: additional robots, sensors, or perception models can be integrated as new services without re-architecting the existing pipeline, providing a credible path from laboratory prototype toward a scalable deployment of the kind increasingly demanded by modern e-commerce warehousing (13) and digital-twin-enabled factories (15).

7 Comparison of Project Plan and Execution

Phase	Planned Objectives	Actual Outcome
Initialization	Design workflows, assign responsibilities, choose appropriate hardware and software, and establish the scope of the warehouse automation system.	Alignment among team members and establishment of workflow for the project was accomplished. Necessary hardware elements were prepared. Initial knowledge regarding the overall system design was developed.
Object Detection Module	Create image datasets, annotate samples, train a YOLO-based model, and implement real-time object detection for identifying items in the workspace.	Image data sets were collected and annotated successfully. The YOLO model was trained and implemented. Accurate object detection was obtained using bounding boxes and confidence values.
Hardware Control System	Design the motor control system for axes X, Y, and Z by using Raspberry Pi GPIO and relay modules. Integrate electromagnetic control for pick-and-place operations.	The motor control circuit with relays was successfully designed and implemented. All three axes have bidirectional movements. Electromagnetic pick-and-place mechanisms were incorporated into the design.
System Integration	Connect object detection outputs with motion commands, perform coordinate mapping, and enable autonomous pick-and-place functionality.	Object detection was linked to motion control. The prototype successfully achieved, motion towards target position, object picking via electromagnet.
Simulation	Design a virtual model of the gantry robot in CoppeliaSim, where testing of its movements, gripping actions, and interactions within the workspace will be conducted before hardware deployment.	A three axis Cartesian robot model was created using CoppeliaSim. Movement scripts and magnetic gripper logic was successfully applied to enable simulation of pick-and-place operations.
Dashboard and Monitoring	Develop a user interface for monitoring system status, inventory levels, order progress, and machine activity in real time.	The Streamlit UI along with FastAPI API was developed. Users could track total number of orders, number of completed orders, number of pending jobs, status of inventory, and machine status via a live interface.
Testing and Optimization	Identify and solve issues related to calibration, motion errors, communication delays, sensor reliability, and simulation instability.	Several problems, such as coordinate mismatch, hardware connections, script errors, and simulation hierarchy errors, were fixed. The stability of the entire system was greatly improved.
Finalization	Prepare the documentation, the technical report, the screenshots, the presentations, and usability analysis of the prototype developed.	Complete preparation of documentation, which included hardware design, software modules, simulation results, dashboard screenshots, and concluding remarks of the project was successfully prepared.

Table 3: Project phases with planned objectives and actual outcomes.

8 Usability of the Results

The developed Fischertechnik crane and inventory management system demonstrates a highly usable prototype of an automated warehousing process. The combination of computer vision, robotics, simulation, and supervision technologies in one complex makes it quite evident that the benefits gained from such a solution are numerous. The practical usability of the results obtained is considered below.

- **Operational Efficiency:** The prototype automates tasks that involve repetitive activities like detecting objects, picking up objects, moving objects, and placing them. Manual handling is less efficient compared to the automation of repetitive tasks as it minimizes human intervention and standardizes the process. The use of YOLO-based object detection enables automatic recognition of items, while the three-axis gantry robot performs repeatable pick-and-place operations. Automation systems of this type are widely used to improve warehouse throughput and reduce repetitive labor demands.
- **Detection Performance and Model Usability:** The object detection model achieved high accuracy suitable for prototype-level operation. The mAP@50 score was 0.977, while the mAP@50–95 score was 0.974. Precision was recorded at 0.885, while Recall was 0.984. Inference required approximately 5.3 ms per image. Although the dataset used during training was small (100 images, 189 instances), data augmentation enabled the model to detect objects accurately for robot automation.
- **Inventory Visibility and Order Monitoring:** The Streamlit dashboard together with the FastAPI backend improved system transparency by allowing users to monitor total orders, completed orders, pending tasks, machine activity, and inventory levels in real time. This increased usability by providing immediate operational feedback and better control of the complete workflow.
- **Decision Support and Supervision:** The dashboard provided a centralized interface where users could supervise operations without direct hardware interaction. Real-time monitoring supported faster decisions, easier fault detection, and improved workflow management. Such digital visibility is essential in modern warehouse environments.
- **Verification using Simulation:** Use of CoppeliaSim for simulating the operation of robots and their motions has enabled the developers to validate their logic through simulation and thus reduce the risks associated with software development. Digital twin simulations have become very popular in the field of warehouses due to these advantages.
- **Low-Cost Educational and Industrial Relevance:** Raspberry Pi, relay modules, Fischertechnik kits, software tools, and simulation capabilities enable this solution to be cost-effective in laboratory settings and research applications. As a result, this project has significance not only as a working model but also as a robotic tool in education.

9 Future Improvements

Although the developed prototype demonstrates the integration of computer vision, robotic manipulation, simulation, and inventory monitoring, several improvements can further enhance the system performance and industrial applicability.

One potential improvement is the fine-tuning of the OCR extraction process to improve the readability and recognition accuracy of serial identification numbers present on warehouse objects. This would enable more reliable object tracking and inventory management within the warehouse environment.

Another key improvement that can be made is trajectory planning algorithms. At present, the robot operates based on pre-defined motion sequences for picking and placing objects. Trajectory planning would allow the robot to generate smoother and more efficient movement paths, while reducing unnecessary motion and improving operational efficiency.

Future work may also focus on real-time synchronization between the physical hardware system and its digital twin in CoppeliaSim. Such synchronization would enable continuous monitoring, validation, and testing of robot behavior through a unified virtual representation of the system.

Additionally, AI-based warehouse optimization techniques can be incorporated to improve storage allocation, task scheduling, inventory handling, and overall warehouse efficiency. Intelligent decision-making algorithms could further enhance the scalability and adaptability of the system in larger warehouse environments.

These improvements would contribute toward the development of a more robust, intelligent, and industrially deployable warehouse automation solution.

10 Conclusions

The project successfully demonstrates an integrated warehouse automation prototype using Fishertechnik crane combining computer vision, robotic manipulation, simulation, and real-time monitoring. The final system establishes a complete workflow that combines object detection, robotic manipulation, monitoring, and digital twin validation into a unified automation framework.

Key outcomes of the project include:

- **Integrated perception and robotic control:** The system achieves successful integration between YOLO computer vision technology and the 3-axis gantry robot, thus ensuring that automatic detection of objects results in corresponding pick-and-place procedures. It demonstrates the feasibility of integrating visual perception with robotic actuation in warehouse automation systems.
- **Automated pick-and-place:** The developed prototype performs automated object pickup, transport, and placement using relay-controlled DC motors and an electromagnetic gripper. The coordinated X, Y, and Z axis movements demonstrate the feasibility of low-cost robotic material handling systems.

- **Simulation Validation:** Simulation through CoppeliaSim provided a means of testing the robot movement algorithm, gripper action, and other interactions in a simulated environment before actual physical implementation. It greatly simplified the process of debugging the system during the development phase.
- **Real-time Monitoring and Visualization:** The combination of Streamlit frontend and FastAPI backend allows for real-time tracking of operations in the system, including the status of inventories, orders, and process execution. The web-based interface improves system supervision and simplifies interaction between the operator and the automation system.
- **Evaluation of model performance:** The object detection model performed at a high level, achieving an mAP@50 of 0.977, an mAP@50–95 of 0.974, Precision of 0.885, and Recall of 0.984. The inference time was found to be approximately 5.3 ms per image. Despite the small number of images in the dataset, data augmentation enabled the system to perform well in terms of localization of objects and execution of automatic pick-and-place procedures.
- **Improved workflow and automation capability:** The prototype reduces manual intervention in repetitive warehouse operations by automating object recognition, movement, and placement processes. The integration of perception, actuation, and monitoring improves operational consistency and demonstrates the advantages of intelligent automation systems.
- **Future scope and scalability:** The design allows for future scalability including using larger datasets for training, ONNX optimizations for inference, trajectory planning algorithms, closed-loop motor control systems, and warehouse management strategies with enhanced AI capabilities. More advanced sensing technology and calibration techniques could be employed to increase accuracy and industrial feasibility.

Overall, the project demonstrates how integrating computer vision, robotic control, simulation, and real-time monitoring can improve automation, operational efficiency, and system intelligence in modern warehouse environments. The developed prototype serves as a scalable foundation for future intelligent warehouse automation research and industrial applications.

11 Source Code Repository

The complete source code for this project—including the object detection training scripts, the data augmentation pipeline, the Raspberry Pi motor-control logic, the CoppeliaSim simulation scene and Lua control scripts, and the FastAPI/Streamlit dashboard—is publicly available at the following repository:

https://github.com/Sourya2000/Fischertechnik_Pick_and_Place_3D_Robot

References

- [1] Z. Huang, Y. Shen, J. Li, M. Fey, and C. Brecher, “A survey on AI-driven digital twins in Industry 4.0: Smart manufacturing and advanced robotics,” *Sensors*, vol. 21, no. 19, p. 6340, 2021.
- [2] S. Surati, S. Hedao, T. Rotti, V. Ahuja, and N. Patel, “Pick and place robotic arm: A review,” *International Research Journal of Engineering and Technology (IRJET)*, vol. 8, no. 2, pp. 2121–2125, 2021.
- [3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788, 2016.
- [4] J. Redmon and A. Farhadi, “YOLO9000: Better, faster, stronger,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 6517–6525, 2017.
- [5] J. Redmon and A. Farhadi, “YOLOv3: An incremental improvement,” *arXiv preprint arXiv:1804.02767*, 2018.
- [6] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal speed and accuracy of object detection,” *arXiv preprint arXiv:2004.10934*, 2020.
- [7] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, “YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 7464–7475, 2023.
- [8] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, “SSD: Single shot multibox detector,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 21–37, 2016.
- [9] G. Jocher and J. Qiu, “Ultralytics YOLO11.” <https://github.com/ultralytics/ultralytics>, 2024. Computer software, AGPL-3.0 License.
- [10] X. Li, W. Wang, L. Wu, S. Chen, X. Hu, J. Li, J. Tang, and J. Yang, “Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 21002–21012, 2020.
- [11] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, “Distance-IoU loss: Faster and better learning for bounding box regression,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, pp. 12993–13000, 2020.
- [12] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.
- [13] R. Bogue, “Growth in e-commerce boosts innovation in the warehouse robot market,” *Industrial Robot: An International Journal*, vol. 43, no. 6, pp. 583–587, 2016.

- [14] E. Rohmer, S. P. N. Singh, and M. Freese, “V-REP: A versatile and scalable robot simulation framework,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 1321–1326, 2013.
- [15] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, “Digital twin in industry: State-of-the-art,” *IEEE Transactions on Industrial Informatics*, vol. 15, no. 4, pp. 2405–2415, 2019.
- [16] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The PASCAL visual object classes (VOC) challenge,” *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303–338, 2010.
- [17] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: Common objects in context,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 740–755, 2014.
- [18] C. Shorten and T. M. Khoshgoftaar, “A survey on image data augmentation for deep learning,” *Journal of Big Data*, vol. 6, no. 1, pp. 1–48, 2019.
- [19] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [20] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *International Conference on Learning Representations (ICLR)*, 2015.
- [21] ONNX Community, “ONNX: Open neural network exchange.” <https://github.com/onnx/onnx>, 2019. Computer software.
- [22] Streamlit Inc., “Streamlit: A faster way to build and share data apps.” <https://github.com/streamlit/streamlit>, 2019. Computer software.
- [23] S. Ramírez, “FastAPI: Modern, fast web framework for building apis with Python.” <https://github.com/fastapi/fastapi>, 2018. Computer software.
- [24] Jaied AI, “EasyOCR: Ready-to-use ocr with 80+ supported languages.” <https://github.com/JaiedAI/EasyOCR>, 2020. Computer software.
- [25] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, 2017.
- [26] B. Shi, X. Bai, and C. Yao, “An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298–2304, 2017.